PHP end-of-term exam (2022-23-1)

Due No Due Date **Points** None

Important information

- You have **180 minutes** to complete the exam. Filling in the included `README.eng.md` template, reading the tasks, downloading the materials, compressing and uploading your solution is **included** in the time limit.
- Solutions must be uploaded to ELTE Canvas. **The system closes at 17:00 (Hungarian time) and there will be no opportunity to submit your solution after the aforementioned time**.
- You may use **any kind of material** (including documentation, lectures, class exercises, cheat sheets, etc) during the exam, **except human help** (synchronous, asynchronous, chat, forums, etc)! By filling in the `README.eng.md` file, you accept the possible consequences of cheating.
- The tasks are independent and can be solved **in any order**.
- To begin your work you should first **download the starting package we have prepared**. Each tasks has its own directory in the package. All tasks are somewhat prepared. Other than the provided files, you are free to use any other files (to store data, organize your code, implement AJAX/fetch), but make sure your solutions are in the correct folder and can be started with the `index.php` file!
- The starting package contains a template called `README.eng.md` where you should fill in your name and Neptun code (the <> characters must be removed). **Submissions without a properly filled `README.eng.md` file will not be graded!**
- Each directory contains a file named `TASKS.md`. Replace the space in `[ ]` with an `x` where you have (even partially) succeeded to complete the task. This will make us easier to find your solution when we grade your exam.

Task 1: Candies (1-candies, 8 pts)

Christmas is almost here, and soon - recalling our childhood - we can climb under the tree to get some candy from the tree and eat it in the back corner. That is why it is important what kind of candy we buy. In the `index.php` file, you will find the registry of a shop selling candies. The `$shop` array contains the brand, the type, and the price of a candy available in the shop. Based on this, you need to generate the following table:

- a. (2 pts) In the first row of the table, list the **different** types of candy (`type`) sold in the store as a table header!
- b. (1 pt) In the first column of the following rows of the table, list the **different** brands of candy sold in the store (`brand`) as table headers!
- c. (1 pt) Sort the types and the brands in ascending alphabetical order!
- d. (2 pts) Write the corresponding price in a table cell determined by the brand and type!
- e. (1 pt) Give the table cell, containing the price, the `lowest` or `largest` style class, if the given price is the least or most expensive in the table, respectively!

- f. (1 pt) In the last column of the table, show the average price of the types of candies from the given brand!

Help: you can use `array_unique` *(https://www.php.net/manual/en/function.array-unique.php) and* `array_column` *(https://www.php.net/manual/en/function.array-column) functions for a more efficient solution!*

Task 2: You've just won a phone! (2-phone, 8 pts)

Surely everyone has come across a phishing website at some point which claimed that we are the lucky one millionth visitor and we've just won a new phone. All we need do to in order to put that new phone under the Christmas tree is to enter our credit card details. Your task is to validate the form **on server side** (therefore HTML level validation is not satisfactory) used on such a page according to the conditions below. Hopefully the winner won't find an unplesant surprise after filling in everything correctly.

- a. (1 pt) The `fullname` field is required and must contain at least two words.
- b. (1 pt) The `email` field is required and must contain a formally valid e-mail address.
- c. (1 pt) The `colour` field is required and its value can only be `black`, `white`, `gold`, `pink` or `blue`.
- d. (1 pt) The `cardnumber` (credit card number) field is required and must be exactly 19 characters long.
- e. (1 pt) The `cardnumber` (credit card number) field must follow the `XXXX-XXXX-XXXX-XXXX` format where the `X` characters replaced by digits.
- f. (1 pt) In case of incorrect input, an error message should appear next to the given field, informing about the cause of the error.
- g. (1 pt) The form placed on the page should keep its state which means that the submitted values should be written back into the fields.
- h. (1 pt) The `div` with ID `success` and its contents should only appear in the generated HTML code if the form was submitted by the user and no errors were found during validation.

Task 3: Gift list (3-gift-list, 8 pts)

Christmas preparations include buying presents. In larger families, however, it can be difficult to keep track of what we will give to whom and in what condition it is (we ordered it, where to go for it, whether we have it or it was a bad idea, etc.). Let us create a PHP application that will help us manage this. We should be able to add family members, record ideas per family member, and add comments to the idea, as well as modify the status of the idea. The idea has two statuses: `active`, i.e. the idea is valid or it has been rejected; `ready`, i.e. we have some work with the idea, or we can finish dealing with it.

The data must be stored in a file, the storage format is up to you. You may use the `Storage` class shown in the lectures but it's not obligatory. If we store the data in JSON format, one possible storage method is to store the family members in one JSON file and the ideas in another JSON file, and the comments within the ideas.

- a. (1 pt) With the help of the form on the `index.php` page, we should be able to add a new family member! *Note: the form does not need to be validated on server side in this task, we assume that it is filled in correctly!*
- b. (1 pt) List the stored family members on the `index.php` page! Each family member should have a link that points to the `member.php` page, passing the ID of the family member in the URL!
- c. (1 pt) On the `member.php` page, write the name of our selected family member in the header!
- d. (1 pt) Use the *"New idea"* form on the `member.php` page to add a new idea for the given family member! For an idea, we store the name of the idea, the activity of the idea (`active`, by default true), the state of the idea (`ready`, false by default), and the comments to it (empty array by default). The form does not need to be validated on the server side.
- e. (1 pt) On the `member.php` page, list the ideas belonging to the given family member and the comments within the ideas!
- f. (1 pt) Make it possible to add a new comment to the idea using the form within the idea! The form does not need to be validated on the server side! You can store the ID of the idea, for example, in a hidden field in the form.
- g. (1 pt) By clicking on the *"Complete"* or *"Hide"* button within the idea, change the `ready` state of the given idea to true or the `active` state to false, respectively! Show a checkmark next to the idea if it is ready, and inactive ideas should not be listed!
- h. (1 pt) On the `index.php` page, next to the family members, display how many of the active ideas are ready! (in the format of `(ready active) / (all active)`)



Task 4: Recipe tracker (4-recipes, 8 pts)

Baking and cooking are natural parts of the festive mood. In this task you need to create an application which helps us keep track of what ingredients we have in the fridge **using a session** to bake something. The starting package already contains the file `recipes.json` which stores the list of ingredients needed to prepare some of the most famous Hungarian Christmas dishes. (It might be helpful to analyse and understand the data structure before starting to code.)

- a. (1 pt) Display the names of the known recipes and make them redirect to a details page.
- b. (1 pt) Also display the contents of the fridge that is stored **in a session** during this task. If there is no session started when the page is loaded, initialize the fridge as empty.
- c. (1 pt) On the details page, list all the ingredients with checkboxes which are needed to prepare the chosen recipe. The checkbox should be checked and disabled if the given ingredient is already in the fridge.
- d. (2 pts) Make the list of checkboxes on the details page submittable using a form. Add the items that the user marks checked into the fridge. (Warning! A checked field does not automatically mean that the item needs to be added to the fridge since the checkbox may alredy be checked when the page renders.)
- e. (1 pt) Mark (e.g. by displaying the text *"Can make!"*) those recipes on the main page where every needed ingredient is available in the fridge.

- f. (2 pts) If everything is in the fridge for a recipe, then there should be a way to make the recipe with by clicking on a link or button. Making the recipe means removing all the included ingredients from the fridge.

